

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



4일차

04 VCP-G CAN TASK 구현

Telechips TOPST



Table of Contents

01



ControlBreadBoardSensors() 개요

CAN 메시지별 CAN ID와 Data 값에 의해 Sensor 제어 로직 구현

메시지 종류	ID	Data
Brake Light	0x101	[0] : on-0x01, off-0x02
Turn Signal Light	0x102	[0] : left-0x01, right-0x02 [1] : on-0x01, off-0x02
Emergency Signal Light	0x103	[0] : on-0x01
Head Light	0x104	[0] : on-0x01, off-0x02
Fuel Level	0x105	[0] : 0%-0x00 ~ 100%-0x64
Vehicle Speed	0x106	[0] : 0kph-0x00 ~ 80kph-0x64
Wheel Angle	0x107	[0] : 0(deg)-0x00 ~ 127(deg)-0x7F

```
enum VCP_IO_TYPE {
    VCP_IO_BREAK_LIGHT=0x101,
    VCP_IO_TURN_SIGNAL=0x102,
    VCP_IO_EMER_SIGNAL=0x103,
    VCP_IO_HEAD_LIGHT=0x104,
    VCP_IO_FUEL_LEVEL=0x105,
    VCP_IO_MOTOR_SPEED=0x106,
    VCP_IO_MOTOR_WHEEL=0x107
};

enum VCP_IO_ACTION {
    VCP_IO_ACTION_ON=0x01,
    VCP_IO_ACTION_OFF=0x02
};

enum VCP_IO_SUBTYPE {
    VCP_IO_SUB_LEFT=0x01,
    VCP_IO_SUB_RIGHT=0x02
};
```

- 설정한 CAN ID별로 동작하는 기능들에 대해 디버깅용 문구출력 작업
 - CAN 통신이 활성화 된다면, 디버깅용 문구가 출력되는 것을 예상

VCP-G 빌드 환경 세팅

- VCP-G 디버깅은 USB-C-to-A 케이블로 PC와 연결
 - 유의) 디버깅 통신을 사용하기 위한 유틸리티는 **minicom** 사용
- D03-P01-VCP 환경 설정과 유사하나, 본 과정을 위해선 일부 파일 수정 후 재빌드
 - 다음 장부터 순차적으로 진행

VCP-G 빌드 환경 세팅

- **FreeRTOS 소스코드 다운로드 (WSL에서 수행)**
 - FreeRTOS Repository edu 폴더로 클론
 - `$ cd`
 - `$ git clone https://github.com/topst-development/FreeRTOS-VCP.git edu`
 - edu 폴더로 이동
 - `$ cd edu`
 - edu/general 브랜치로 체크아웃
 - `$ git checkout edu/general`
 - `$ ./easy-setup_vcp-g.sh`
 - 파일 수정 (vscode or vim)
 - 경로: `~/edu/source/app.sample/app.can.demo/can_vcp_ctrl.c`

VCP-G : can_vcp_ctrl.c 수정

```
void ControlBreadBoardSensors(uint32 mId, uint8 nDataLength, sint8* pucData)
{
    sint8 ucMsgData[2] = {0x00, 0x00};
    if(nDataLength == 0)
    {
        return; // 데이터 없으면 무시
    }
    switch(mId)
    {
        .... // 이 곳에 메시지 수신별 케이스문 작성
    }
}
```

- ControlBreadBoardSensors()의 기본 포맷
 - CAN 메시지 아이디와 데이터를 받는다면
 - 메시지 아이디 별로 분기 처리 (switch – case 문 작성)

VCP-G : can_vcp_ctrl.c 수정

```
case VCP_IO_BREAK_LIGHT: // 브레이크등 0x101
{
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        ControlBrakeLight(TRUE);
    }
    else
    {
        ControlBrakeLight(FALSE);
    }
    break;
}
```

CASE 1

- Brake 등 분기 처리
- 메시지 ON/OFF 에 따라 별도의 함수 처리
 - ControlBrakeLight()

VCP-G : can_vcp_ctrl.c 수정

```

case VCP_IO_TURN_SIGNAL: // 방향등 0x102
{
    ucMsgData[0] = pucData[0];
    ucMsgData[1] = pucData[1];

    if (ucMsgData[0] == VCP_IO_SUB_LEFT)
    {
        if(ucMsgData[1] == VCP_IO_ACTION_ON)
        {
            // turn on the left signal
            ControlSignalLight(TRUE, TRUE);
        }
        else
        {
            // turn off the left light
            ControlSignalLight(TRUE, FALSE);
        }
    }
    if (ucMsgData[0] == VCP_IO_SUB_RIGHT)
    {
        if(ucMsgData[1] == VCP_IO_ACTION_ON)
        {
            ControlSignalLight(FALSE, TRUE);
        }
        else
        {
            ControlSignalLight(FALSE, FALSE);
        }
    }
    break;
}

```

CASE 2

- 방향등 분기 처리
- 왼쪽 지시등 / 오른쪽 지시등 분기 처리
(VCP_IO_SUB_LEFT/VCP_IO_SUB_RIGHT)
- 메시지 ON/OFF 에 따라 별도의 함수 처리
 - ControlSignalLight()

VCP-G : can_vcp_ctrl.c 수정

```
case VCP_IO_EMER_SIGNAL: // 비상등 0x103
{
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        mcu_printf("Emergency Signal ON<->OFF\r\n");
    }
    break;
}
```

CASE 3

- 비상등 처리
- 메시지 수신 시, mcu_print 출력

VCP-G : can_vcp_ctrl.c 수정

```
case VCP_IO_HEAD_LIGHT: // 헤드라이트 0x104
{
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        ControlHeadLight(TRUE);
    }
    else
    {
        ControlHeadLight(FALSE);
    }
    break;
}
```

CASE 4

- 전방 헤드라이트 처리
- 메시지 ON/OFF 에 따라 별도의 함수 처리
 - ControlHeadLight()

VCP-G : can_vcp_ctrl.c 수정

```
case VCP_IO_FUEL_LEVEL: // 연료게이지 0x105
{
    ucMsgData[0] = (pucData[0] > 100)?100:pucData[0];
    ControlFuelLevel(ucMsgData[0]);
    break;
}
```

CASE 5

- 연료게이지 분기 처리
- 메시지 속 데이터 수신시, ControlFuelLevel () 함수 처리

VCP-G : can_vcp_ctrl.c 수정

```
case VCP_IO_MOTOR_SPEED: // 모터 스피드 0x106
{
    sint8 speed = pucData[0]; // 0~80
    mcu_printf("[MOTOR] speed=%d\r\n", (sint8)speed);
    break;
}
```

CASE 6

- 모터 스피드 처리
- 메시지 데이터 수신시 , mcu_print() 출력문 출력

VCP-G : can_vcp_ctrl.c 수정

```
case VCP_IO_MOTOR_WHEEL: // 서보모터 0x107
{
    int8_t input = (int8_t)pucData[0]; // 0~256
    if (input < 0) input = 0;
    if (input > 127) input = 127;
    // 중앙 기준으로 -45~+45도로 매핑
    int angle = 90 + ((input - 65) * 45) / 65;
    mcu_printf("[SERVO] input=%d -> angle=%d deg\r\n", input, angle);
    break;
}

default:
{
    mcu_printf("[%s][%d] undefined can id !!!\r\n", __FUNCTION__, __LINE__);
    break;
}
```

CASE 7

- 서보모터 분기 처리
- 메시지 데이터를 서보 모터에 맞게 input 으로 새로 매핑

Default Case

- can ID 예외처리 문구 출력

VCP-G : can_vcp_ctrl.c 수정

```
/* ----- Brake Light ----- */  
void BrakeLED_ON(void)  
{  
    mcu_printf("Brake LED ON\r\n");  
}  
  
void BrakeLED_OFF(void)  
{  
    mcu_printf("Brake LED OFF\r\n");  
}  
  
void ControlBrakeLight(boolean bTurnOn)  
{  
    if(bTurnOn)  
    {  
        BrakeLED_ON();  
    }  
    else  
    {  
        BrakeLED_OFF();  
    }  
}
```

Brake 등 관련 함수 내용 작성

- ControlBrakeLight()
 ↓
- BrakeLED_ON()
 ↓
- BrakeLED_OFF()
- 순으로 호출

VCP-G : can_vcp_ctrl.c 수정

```
/* ----- Turn Signal Light ----- */
void LeftSignalLED_ON(void)
{
    mcu_printf("Left Signal LED ON\r\n");
}

void LeftSignalLED_OFF(void)
{
    mcu_printf("Left Signal LED OFF\r\n");
}

void RightSignalLED_ON(void)
{
    mcu_printf("Right Signal LED ON\r\n");
}

void RightSignalLED_OFF(void)
{
    mcu_printf("Right Signal LED OFF\r\n");
}
```

```
void ControlSignalLight(boolean bLeft, boolean bTurnOn)
{
    if(bLeft)
    {
        if(bTurnOn)
        {
            LeftSignalLED_ON();
        }
        else
        {
            LeftSignalLED_OFF();
        }
    }
    else
    {
        if(bTurnOn)
        {
            RightSignalLED_ON();
        }
        else
        {
            RightSignalLED_OFF();
        }
    }
}
```

방향지시등 관련 함수 내용 작성

- ControlSignalLight
 - bLeft → LeftSignalLED_ON()/OFF()
 - RightSignalLED_ON()/OFF()

VCP-G : can_vcp_ctrl.c 수정

```
/* ----- Head Light ----- */

void HeadLED_ON(void)
{
    mcu_printf("Head LED ON\r\n");
}

void HeadLED_OFF(void)
{
    mcu_printf("Head LED OFF\r\n");
}

void ControlHeadLight(boolean bTurnOn)
{
    if (bTurnOn) {
        HeadLED_ON();
        mcu_printf("[Head Light] ON\r\n");
    } else {
        HeadLED_OFF();
        mcu_printf("[Head Light] OFF\r\n");
    }
}
```

헤드라이트 관련 함수 내용 작성

- ControlHeadLight()
↓
- HeadLED_ON()
↓
- HeadLED_OFF()
- 순으로 호출

VCP-G : can_vcp_ctrl.c 수정

```

/* ----- Fuel Level ----- */

void lcd_send(uint8 mode, uint8 data)
{
}

void lcd_cmd(uint8 cmd)
{
}

void lcd_data(uint8 dat)
{
}

void lcd_init(void)
{
}

void lcd_print(const char *str)
{
}

void ControlFuelLevel(uint8 fuelLevel)
{
    uint8 pct = (fuelLevel > 100) ? 100 : fuelLevel;
    mcu_printf("[FUEL] Controlling Fuel Level : %d\r\n", pct);
}

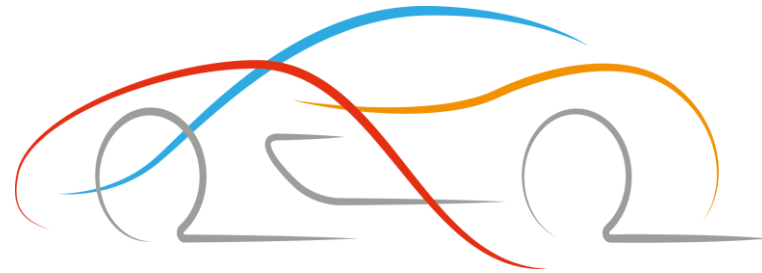
```

연료게이지 관련 함수 내용 작성

- lcd 관련 함수는 모형차와 함께 구현 예정
- ControlFuelLevel()
 - 데이터를 0%-100%로 다시 맵핑
 - 데이터를 받아 출력 구현으로 대체

빌드 & FWDN

- build 진행
 - `$ cd ~/edu/build/tcc70xx/gcc`
 - `$ make clean && make`
- VCP-G 보드와 PC와 연결 (D02-T03-VCP 환경설정 강의자료 참고)
- VCP-G 보드 FWDN 모드 진입
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/edu/tools/fwdn_vcp/fwdn`
`--fwdn ~/edu/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/edu/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`



Thank You